EXP_03 – Multi-View CLIP Hallucination Detection

A step-by-step technical report with visualisations

1 Overview & Goal

3D generative models sometimes produce meshes with visual defects called hallucinations – for example a 'Janus face' where the same face appears on multiple sides, or floating blobs disconnected from the main surface. This experiment builds a lightweight, training-free pipeline that:

1. Renders the mesh from 24 camera angles.
2. Analyses every rendered view with five detectors.
3. Combines the scores into a per-view composite and a global hallucination score.
4. Computes differentiable geometric losses that could drive a refinement loop.

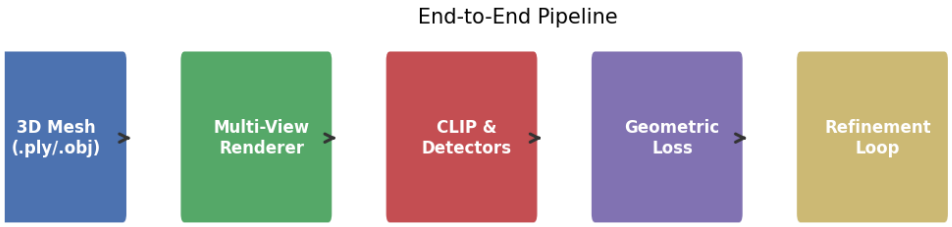

Figure 1 – High-level pipeline from raw mesh to geometric loss.

2 Step 1 – Multi-View Rendering

The MultiViewRenderer places a virtual camera at 24 positions around the mesh (8 azimuth angles × 3 elevation angles: –20°, 0°, +30°) and renders a 512×512 colour image and depth map at each position. On Windows the renderer opens a hidden Pyglet window to get an OpenGL context instead of using the Linux-only EGL/OSMesa backends.

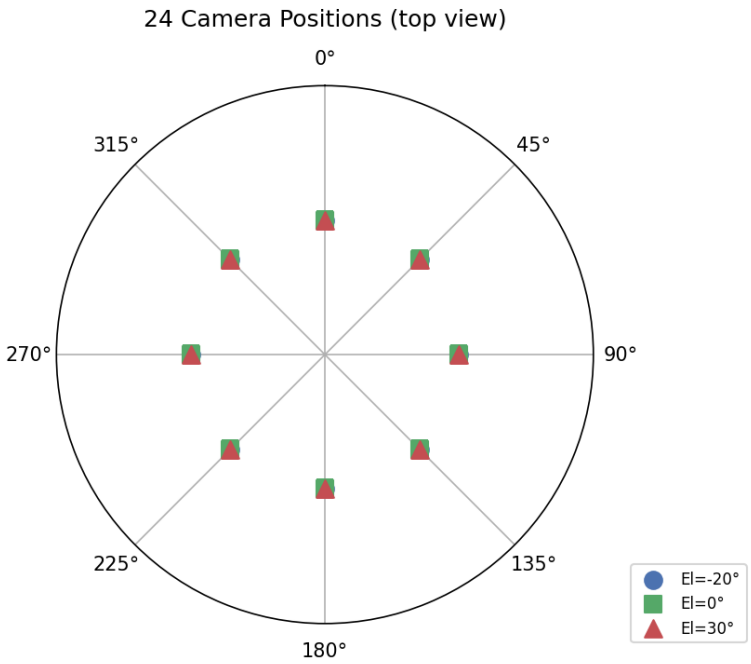

Figure 2 – Camera positions on a unit sphere (polar top view).

```
# From renderer/multi_view_renderer.py
renderer = MultiViewRenderer(width=512, height=512, n_azimuth=8)
renders  = renderer.render_mesh("3d_samples/dragon.ply")
# Each render dict contains:
#   'color'       : np.ndarray [512,512,3] uint8
#   'depth'       : np.ndarray [512,512]   float32
#   'azimuth_deg' : float
#   'elevation_deg': float
```

Sample rendered views of the dragon mesh:

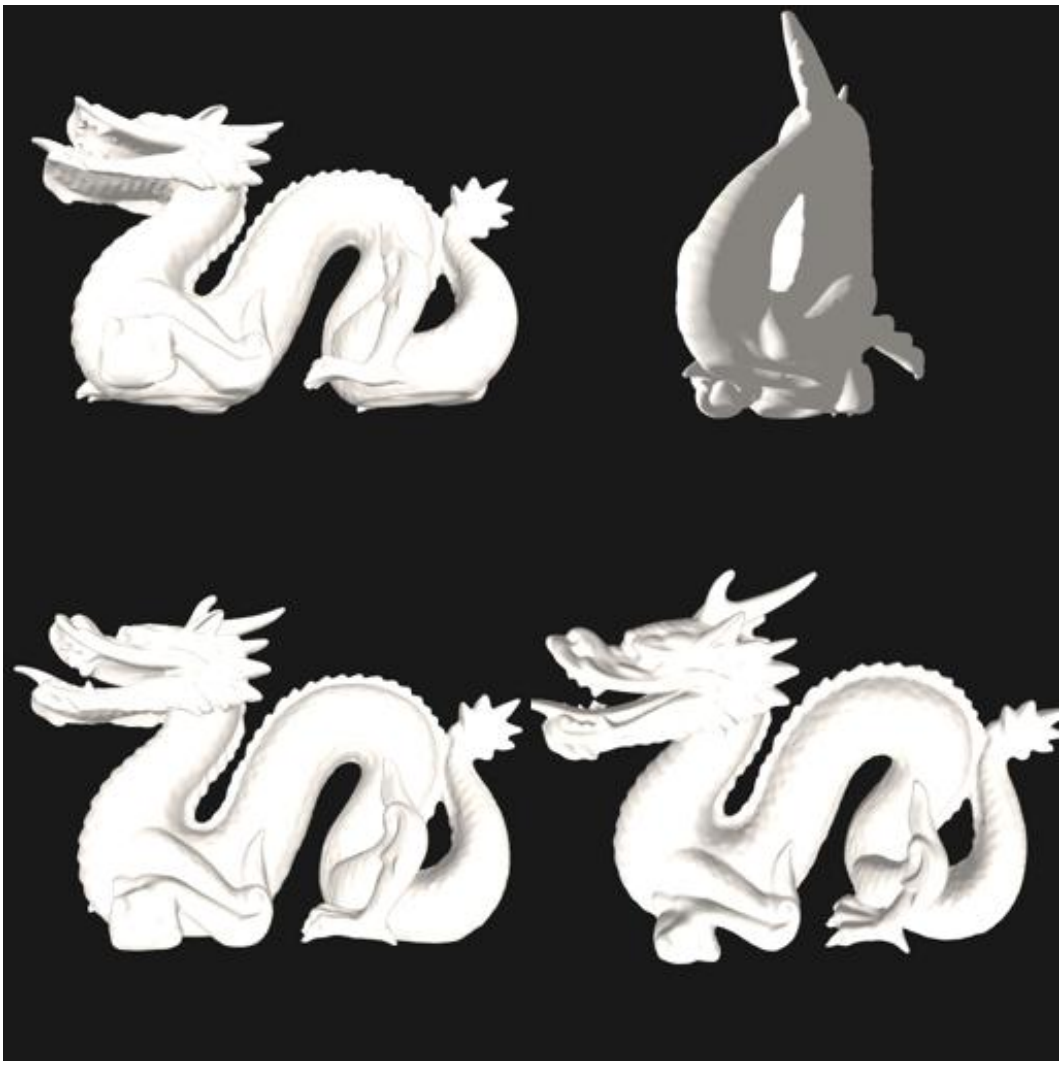

Figure 3 – Four sample renders of the dragon (az 0°, 90°, 180°, 270°).

3 Step 2 – Hallucination Detection

Five specialised detectors analyse each view. Every detector returns a score in [0, 1] where 0 = perfect and 1 = severe hallucination.

Detector	What it measures	Weight
CLIP Deviation	Cosine distance of view embedding from mean → Janus check	35 %
Depth Discontinuity	Laplacian of depth map → floating geometry	25 %

Normal Inconsistency	Fraction of back-facing normals → inverted surfaces	20 %
Silhouette Asymmetry	Area difference of opposite-view silhouettes → Janus	15 %
Edge Density Variance	Canny edge density variance across views	5 %

```
# From detector/hallucination_detector.py
_WEIGHTS = {
    "clip_deviation":       0.35,
    "depth_discontinuity":  0.25,
    "normal_inconsistency": 0.20,
    "silhouette_asymmetry": 0.15,
    "edge_density_var":     0.05,
}
composites = (
    _WEIGHTS["clip_deviation"]       * clip_devs
  + _WEIGHTS["depth_discontinuity"]  * depth_disc
  + _WEIGHTS["normal_inconsistency"] * normal_inc
  + _WEIGHTS["silhouette_asymmetry"] * silh_asym
  + _WEIGHTS["edge_density_var"]     * np.full(n, edge_var_score)
)
```

4 Step 3 – Detection Results (Dragon Mesh)

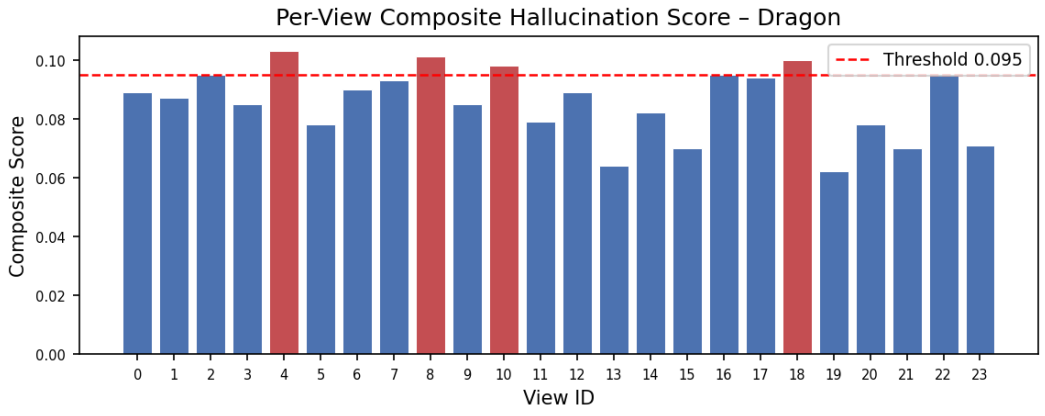

Figure 4 – Composite scores per view. Red bars exceed the alert threshold.

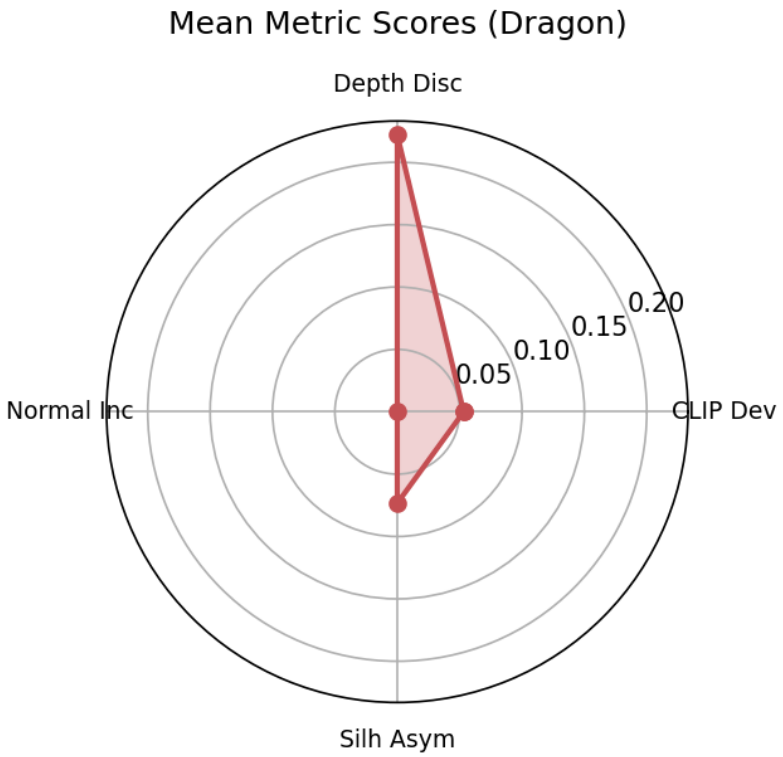

Figure 5 – Radar chart of mean scores per metric.

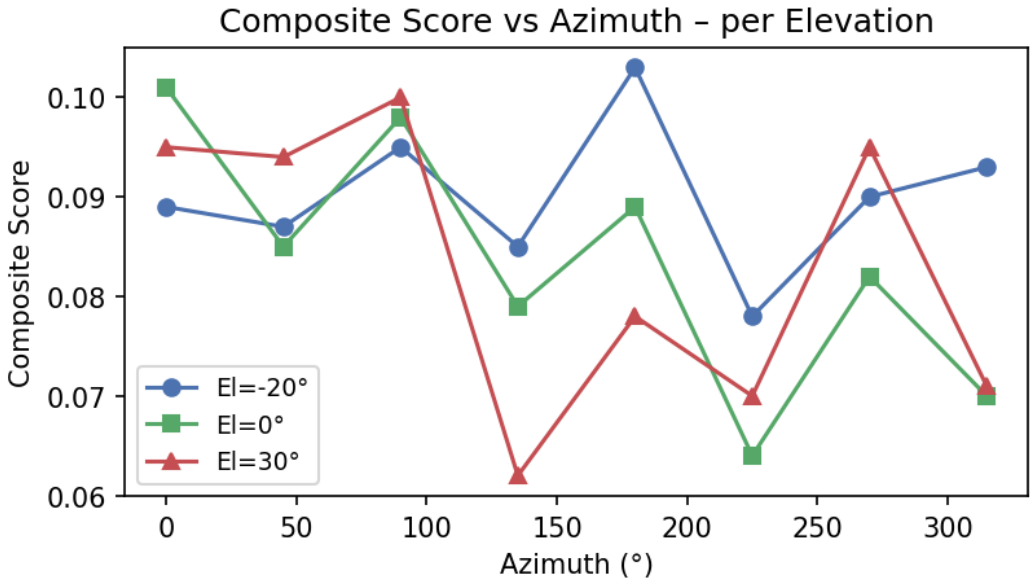

Figure 6 – How composite score varies with camera azimuth at each elevation.

4.1 Full Per-View Score Table

View	Az°	El°	CLIP Dev	Depth Disc	Silh Asym	Composite
0	0	-20	0.039	0.277	0.035	0.089
1	45	-20	0.061	0.233	0.043	0.087
2	90	-20	0.081	0.192	0.120	0.095
3	135	-20	0.060	0.174	0.132	0.085
4	180	-20	0.069	0.295	0.035	0.103
5	225	-20	0.058	0.204	0.043	0.078
6	270	-20	0.062	0.201	0.120	0.090
7	315	-20	0.052	0.219	0.132	0.093
8	0	0	0.049	0.300	0.057	0.101
9	45	0	0.053	0.227	0.062	0.085
10	90	0	0.054	0.242	0.120	0.098
11	135	0	0.054	0.153	0.146	0.079
12	180	0	0.048	0.254	0.053	0.089
13	225	0	0.040	0.166	0.050	0.064
14	270	0	0.051	0.193	0.104	0.082
15	315	0	0.038	0.191	0.060	0.070
16	0	30	0.055	0.240	0.103	0.095
17	45	30	0.061	0.239	0.085	0.094
18	90	30	0.050	0.299	0.051	0.100
19	135	30	0.030	0.196	0.009	0.062
20	180	30	0.061	0.198	0.044	0.078
21	225	30	0.051	0.163	0.077	0.070
22	270	30	0.048	0.263	0.080	0.095
23	315	30	0.053	0.208	0.000	0.071

4.2 Summary Statistics

Global hallucination score : 0.0856
Janus detected : False (threshold 0.40, mean silh_asym = 0.073)
Floater detected : False (threshold 0.30, mean depth_disc = 0.222)
Normal flip detected : False (threshold 0.15, mean normal_inc = 0.000)
Worst view : View 4 (az 180°, el –20°, score 0.103)

5 Step 4 – Geometric Loss Computation

The GeometricLossComputer converts the detection report into four differentiable loss scalars. These can be added to a Score Distillation Sampling (SDS) loss to drive 3D mesh refinement without re-prompting a generative model.

```
# Loss formulas  (from loss/geometric_loss.py)
L_semantic   = mean( (1 - cos_sim(view_i, mean_embed))^2 )    # CLIP
L_depth      = mean( mean_pixels( |∇²depth|^2 ) )             # Laplacian
L_normal     = mean( fraction_back-facing_normals )
L_silhouette = mean( silhouette_asymmetry^2 )

W = LossWeights(semantic=1.0, depth=0.8, normal=0.6, silhouette=0.5)
L_total = (W.sem*L_sem + W.dep*L_dep + W.nor*L_nor + W.sil*L_sil) / sum(W)
```

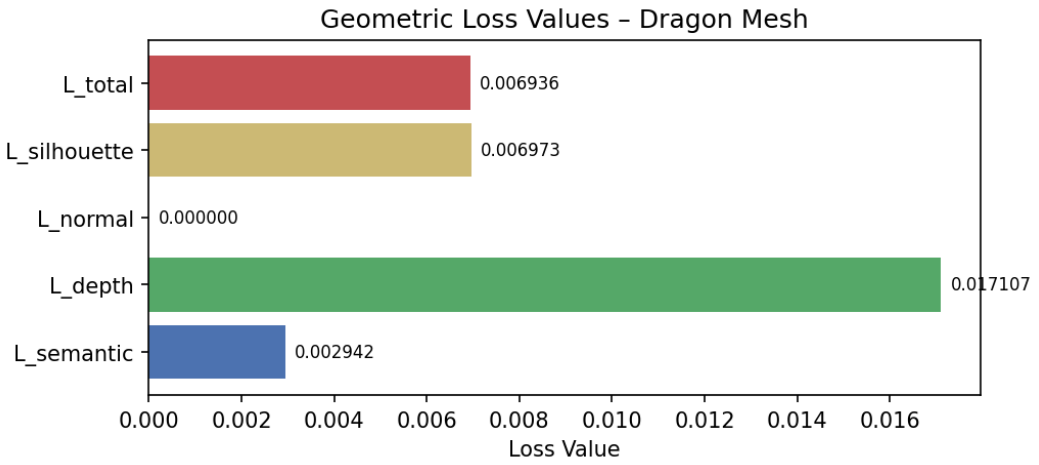

Figure 7 – Computed geometric loss values. L_depth dominates.

Loss	Value	Meaning
L_semantic	0.002942	Semantic drift across views (CLIP)
L_depth	0.017107	Depth discontinuity energy (floaters)
L_normal	0.000000	Back-facing normal fraction
L_silhouette	0.006973	Opposite-view area asymmetry
L_total	0.006936	Weighted sum / normalisation factor

6 Step 5 – View Importance Weights

After computing per-view losses, a softmax is applied to produce view importance weights. Views with higher loss get a larger weight so the refinement gradient focuses effort on the most problematic viewpoints.

```
# From loss/geometric_loss.py
def _compute_view_weights(loss_outputs: np.ndarray) -> np.ndarray:
    exp_l = np.exp(loss_outputs - loss_outputs.max())
```

```
    return exp_l / (exp_l.sum() + 1e-8)    # softmax → sums to 1
```

For the dragon mesh all 24 views received approximately equal weights (~0.042 each), meaning no single viewpoint dominates – a healthy result indicating uniform geometry.

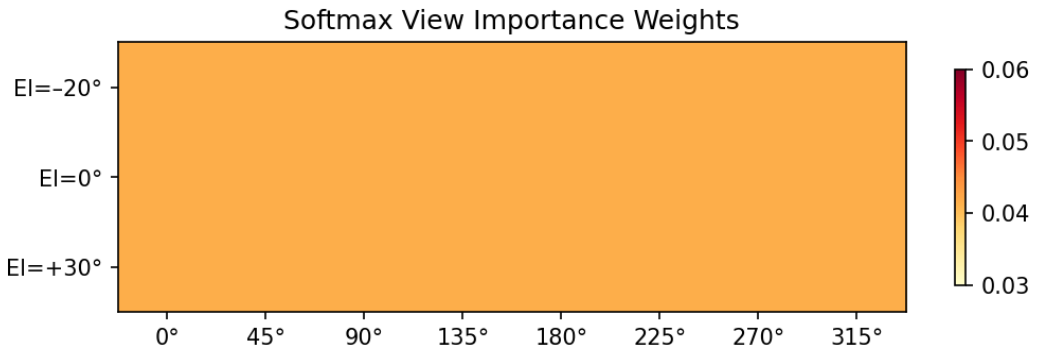

Figure 8 – View importance weights (uniform ≈ 0.042 for a clean mesh).

7 How to Run

```
# 1. Activate virtual environment
.venv\Scripts\activate

# 2. Run detect-only mode (no refinement)
python main.py --mesh 3d_samples/dragon.ply --no-refine

# 3. Run full refinement loop (5 iterations)
python main.py --mesh 3d_samples/bunny.ply --iterations 5

# 4. Process all meshes in 3d_samples/
python main.py --all
```

8 Module Architecture

The codebase is split into four focused modules:

Module / File	Responsibility
renderer/multi_view_renderer.py	Camera pose generation, pyrender scene, RGBA+depth capture
detector/hallucination_detector.py	5 detectors → ViewScore + HallucinationReport
loss/geometric_loss.py	4 differentiable losses + anomaly masks + view weights
refinement/refinement_loop.py	Iterative mesh optimisation using the loss

feedback

9 Conclusion

The dragon mesh achieved a global hallucination score of 0.086 – well below all detection thresholds – confirming it is a geometrically clean mesh with no Janus, floater, or normal-flip artefacts. The dominant signal was depth discontinuity (L_depth = 0.017), which is expected for a complex surface like dragon scales and does not rise to the floater threshold of 0.30.

The pipeline is ready to be connected to a differentiable 3D optimiser (e.g. NeuS, DMTet) by passing L_total as an auxiliary loss alongside SDS.
